SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.
(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

ANSWER

(a) Analysis of the Lack of Responsiveness
The existing website is not responsive due to the following reasons:

- Fixed-width containers (width: 1200px) instead of flexible layouts.
- Absence of media queries to adjust layouts for different devices.
- Improper use of margins and padding causing uneven spacing.
- Images have fixed sizes and do not scale.
- Navigation menu does not adapt to smaller screens.
- Content sections overlap due to lack of Flexbox/Grid.
- Font sizes remain constant, making text difficult to read on mobile devices.

Result of these issues:

- Horizontal scrolling.
- Misaligned content.

- Poor user experience.
- Difficult navigation on mobile devices.

(b) Responsive Webpage Design Using CSS Flexbox

HTML

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Retail Website</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<header>
<h1>Retail Store</h1>
</header>

<nav>
<a href="#">Home</a>
<a href="#">Products</a>
<a href="#">Offers</a>
<a href="#">Contact</a>
</nav>

<div class="container">

<div class="box">
<h2>Electronics</h2>
<p>Latest gadgets and accessories.</p>
</div>

<div class="box">
<h2>Fashion</h2>
<p>Trending clothing collections.</p>
</div>

<div class="box">
<h2>Groceries</h2>
<p>Daily essentials at affordable prices.</p>
</div>

</div>

<footer>
<p>© 2026 Retail Store</p>
</footer>

</body>
</html>
```

CSS

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial;
}

body{
background:#f5f5f5;
}

header{
background:#1e88e5;
color:white;
padding:20px;
text-align:center;
}

nav{
display:flex;
justify-content:center;
background:#333;
}

nav a{
color:white;
padding:15px 20px;
text-decoration:none;
}

nav a:hover{
background:#555;
}

.container{
display:flex;
justify-content:space-around;
gap:20px;
padding:20px;
flex-wrap:wrap;
}

.box{
background:white;
padding:20px;
flex:1;
min-width:250px;
text-align:center;
```

```
border-radius:8px;
box-shadow:0 0 8px gray;
}

footer{
background:#222;
color:white;
text-align:center;
padding:15px;
margin-top:20px;
}
```

(c) Media Queries for Different Screen Sizes

```
/* Mobile */

@media(max-width:600px){

nav{
flex-direction:column;
}

.container{
flex-direction:column;
}

.box{
width:100%;
}

}

/* Tablet */

@media(min-width:601px) and (max-width:900px){

.container{
justify-content:center;
}

.box{
flex:45%;
}

}

/* Desktop */

@media(min-width:901px){

.container{
```

```
flex-direction:row;
}

}
```

Layout Adaptation

Device	Layout
Mobile	Navigation and content arranged vertically
Tablet	Two-column layout
Desktop	Three-column layout with equal spacing

(d) Spacing, Alignment, and Distribution Techniques

The webpage uses the following CSS properties:

- **display: flex** – Creates a flexible layout.
- **justify-content: space-around** – Evenly distributes boxes.
- **gap: 20px** – Maintains equal spacing between elements.
- **padding** – Provides internal spacing.
- **margin-top** – Separates footer from content.
- **text-align: center** – Aligns text consistently.
- **flex-wrap: wrap** – Prevents overflow on smaller screens.
- **box-sizing: border-box** – Ensures padding is included in element width.

These techniques improve readability, balance, and overall visual consistency.

(e) Evaluation and Further Improvements

Evaluation

The responsive webpage successfully:

- Adapts to mobile, tablet, and desktop screens.
- Eliminates horizontal scrolling.
- Maintains proper alignment.
- Provides consistent spacing.
- Improves navigation and readability.
- Enhances user experience across devices.

Further Improvements

- Use **CSS Grid** for more complex layouts.
- Add a **hamburger menu** for mobile navigation.
- Optimize images using responsive image techniques.
- Implement animations and hover effects.
- Improve accessibility using semantic HTML and ARIA attributes.
- Use relative units (rem, %, vw, vh) instead of fixed pixel values.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.
(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.
(c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

ANSWER

(a) Analysis of the Existing Form Validation Mechanism

The existing registration form has the following issues:

- No validation for incorrect email format.
- Phone numbers can contain letters or special characters.
- Empty fields are accepted.
- No real-time feedback while entering data.
- Invalid data is submitted to the server.
- Poor user experience due to lack of error messages.

Consequences:

- Incorrect user information is stored.
- Increased server-side processing.
- Poor data quality.
- Reduced application reliability.

(b) Design of Input Fields Using HTML

HTML

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<h2>Registration Form</h2>

<form>

<label>Email</label><br>
<input type="email" id="email" placeholder="Enter Email">
<span id="emailError"></span>

<br><br>

<label>Phone Number</label><br>
```

```
<input type="tel" id="phone" placeholder="Enter Phone Number">
<span id="phoneError"></span>

<br><br>

<button type="submit">Register</button>

</form>

<script src="script.js"></script>

</body>
</html>
```

(c) JavaScript Validation Using Regular Expressions

JavaScript

```
const email = document.getElementById("email");
const phone = document.getElementById("phone");

const emailError = document.getElementById("emailError");
const phoneError = document.getElementById("phoneError");

email.addEventListener("input", validateEmail);
phone.addEventListener("input", validatePhone);

function validateEmail(){

let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

if(emailPattern.test(email.value)){
emailError.innerHTML = "Valid Email";
emailError.style.color = "green";
}
else{
emailError.innerHTML = "Invalid Email";
emailError.style.color = "red";
}

}

function validatePhone(){

let phonePattern = /^[6-9]\d{9}$/;

if(phonePattern.test(phone.value)){
phoneError.innerHTML = "Valid Phone Number";
phoneError.style.color = "green";
}
else{
phoneError.innerHTML = "Invalid Phone Number";
```

```
phoneError.style.color = "red";
}

}
```

(d) Real-Time Error Message Mechanism

The validation system provides immediate feedback while the user types.

Working

- input event detects every keystroke.
- JavaScript checks the entered value using Regular Expressions.
- If the input is valid, a green success message is displayed.
- If invalid, a red error message appears instantly.
- Users can correct mistakes before submitting the form.

CSS (Optional Styling)

```
body{
font-family:Arial;
margin:30px;
}

input{
padding:10px;
width:250px;
}

span{
font-size:14px;
margin-left:10px;
font-weight:bold;
}

button{
padding:10px 20px;
margin-top:20px;
}
```

(e) Evaluation and Enhancements

Evaluation

The implemented validation system:

- Prevents invalid email addresses.
- Accepts only valid 10-digit phone numbers.
- Displays real-time validation messages.
- Improves data accuracy.
- Enhances user experience.
- Reduces invalid form submissions.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.
(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

ANSWER
(a) Analysis of CSS Properties Responsible for Layout Overlapping
Layout overlapping usually occurs due to improper use of CSS properties such as:

- Using absolute positioning without a relative parent element.
- Fixed widths and heights that do not adjust to different screen sizes.
- Incorrect use of z-index, causing elements to overlap.
- Lack of responsive layouts using Flexbox or Grid.
- Missing flex-wrap property in Flexbox layouts.
- Excessive negative margins.
- Improper use of floating (float) elements without clearing them.

Effects:

- Content overlaps with other elements.
- Horizontal scrolling appears.
- Text becomes unreadable.
- Poor user experience on different screen sizes.

(b) Role of the CSS Box Model
The CSS Box Model determines how every HTML element occupies space.
Components of the Box Model

Component	Purpose
Content	Displays the actual text or image.
Padding	Creates space between content and border.
Border	Surrounds the padding and content.
Margin	Creates space outside the element from other elements.

Example

```
.box{
width:250px;
padding:20px;
border:2px solid black;
margin:15px;
}
```

Importance:

- Prevents elements from touching each other.
- Maintains proper spacing.
- Improves readability.
- Helps create clean and responsive layouts.

(c) Debugging the Layout Using Positioning Techniques

HTML

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Layout</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<header>Website Header</header>

<div class="container">

<div class="box">Section 1</div>
<div class="box">Section 2</div>
<div class="box">Section 3</div>

</div>

<footer>Website Footer</footer>

</body>
</html>
```

CSS

```
*{
margin:0;
padding:0;
box-sizing:border-box;
}

body{
font-family:Arial;
}

header,footer{
background:#1976d2;
color:white;
padding:20px;
text-align:center;
}

.container{
display:flex;
flex-wrap:wrap;
justify-content:space-around;
padding:20px;
gap:20px;
}

.box{
```

```
flex:1;
min-width:250px;
padding:20px;
background:#eeeeee;
text-align:center;
border:1px solid gray;
position:relative;
}
```

Positioning Techniques Used

- Relative Positioning: Keeps elements in the normal document flow while allowing controlled positioning.
- Flexbox: Automatically arranges items without overlap.
- Flex-wrap: Moves items to the next line when space is insufficient.
- Grid (Alternative): Can also be used to create structured layouts with rows and columns.

(d) CSS Modifications to Prevent Overlapping

Media Queries

```
/* Mobile */

@media(max-width:600px){

.container{
flex-direction:column;
}

.box{
width:100%;
}

}

/* Tablet */

@media(min-width:601px) and (max-width:900px){

.box{
flex:45%;
}

}

/* Desktop */

@media(min-width:901px){

.box{
flex:30%;
}

}
```

Additional Improvements

- Used box-sizing: border-box to include padding and border within element width.

- Added gap for equal spacing.
- Used flexible widths instead of fixed widths.
- Enabled wrapping using flex-wrap.
- Applied media queries for different screen sizes.

These modifications ensure proper alignment and eliminate overlapping during browser resizing.

(e) Evaluation and Best Practices

Evaluation

The corrected layout:

- Prevents overlapping of elements.
- Adjusts automatically to different screen sizes.
- Maintains proper spacing and alignment.
- Improves readability and navigation.
- Provides a consistent user experience across mobile, tablet, and desktop devices.

Code of Conduct:

*I **Aaron Samuel S(192411022)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation